

Bases de Datos

Clase 6 y 7: SQL

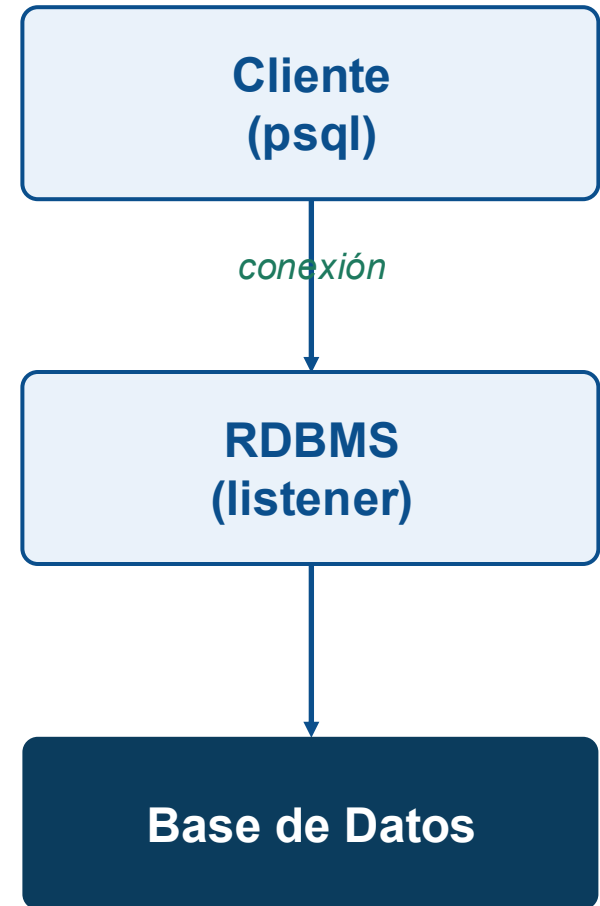
Lo que hemos visto

- Sabemos lo que es un modelo relacional
- Sabemos hacer consultas en Álgebra Relacional
- Sabemos modelar un problema usando el modelo E/R
- Sabemos transformar un MER a un esquema relacional
- Dependencias funcionales y anomalías
- Formas normales

Ahora: SQL

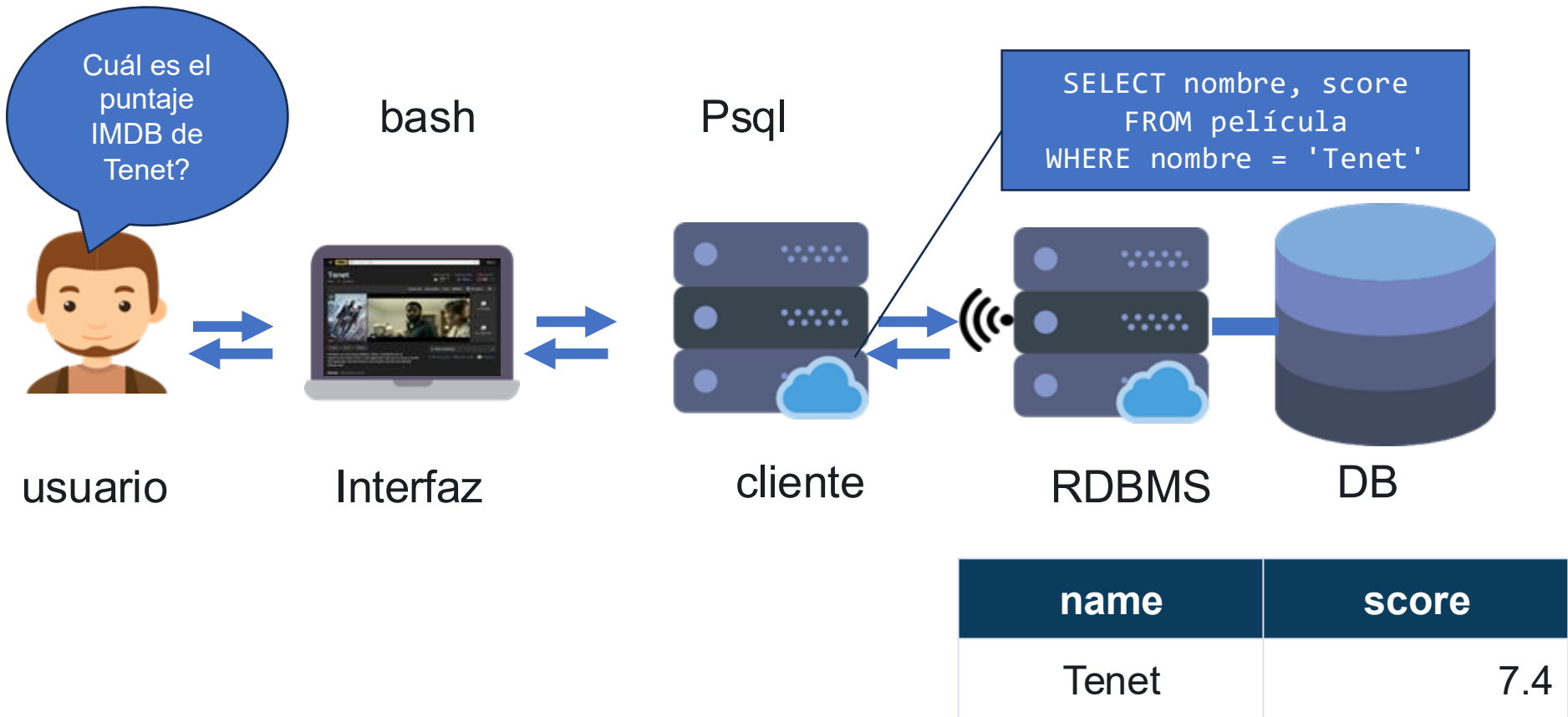
Cómo funciona un RDBMS

- Un DBMS relacional es un programa (p. ej. PostgreSQL) que se instala en un computador (servidor)
- Este programa se mantiene escuchando conexiones (listener)
- El usuario, generalmente otro programa cliente (p. ej. psql), se conecta al RDBMS y le entrega instrucciones



DBMS Relacional

¿Cuál es el puntaje IMDB de Tenet?



name	score
Tenet	7.4

Resultado devuelto al cliente

SQL

Structured Query Language

- Usado para todas las comunicaciones con bases de datos relacionales
- Aplicaciones web, locales, móviles, análisis de datos, etc.
- Incluso algunas arquitecturas serverless lo requieren, o usan lenguajes basados en SQL
- Los sistemas implementan un "subconjunto" del estándar ISO/IEC 9075:2023 — cada uno con diferencias sutiles
- **Lenguaje declarativo, no procedural**

Declarativo vs. Procedural

SQL (declarativo)

- Decimos lo que queremos, sin dar detalles de cómo se computa
- El DBMS transforma la consulta SQL en un algoritmo ejecutado sobre un lenguaje procedural

Python (procedural)

- Para hacer algo debemos indicar, paso a paso, el procedimiento completo

En simple: lo que quiero vs. cómo lo obtengo

SQL: sus sublenguajes

DDL

Lenguaje de Definición de Datos

Crear y modificar tablas, atributos y llaves

DML

Lenguaje de Manipulación de Datos

Manipula los datos almacenados en la base de datos

DCL

Lenguaje de Control de Datos

Gestiona permisos y derechos de acceso de los usuarios

DQL

Lenguaje de Consulta de Datos

Permite realizar consultas y recuperar información

TCL

Lenguaje de Control de Transacciones

Controla las transacciones, asegura la integridad

También existen lenguajes procedurales como PL/SQL, T-SQL, PL/pgSQL, etc.

Tipos de datos

Caracteres (Strings)

- `char(20)` — largo fijo
- `varchar(20)` — largo variable

Números

- `int`, `smallint`, `float`, ...

Tiempo y fecha

- `time` — hora formato 24 hrs.
- `date` — fecha
- `timestamp` — fecha + hora

¡Y varios otros! Dependen del RDBMS que se esté usando.

Creando un esquema

Consideremos de ejemplo:

- Peliculas(id, nombre, año, categoria, calificacion, director)
- Actores(id, nombre, edad)
- Actuo_en(id_actor, id_pelicula)

Sin datos

```
CREATE TABLE Peliculas(
  id int,
  nombre varchar(30),
  año int,
  categoria varchar(30),
  calificacion float,
  director varchar(30)
)
```

id	nombre	año	categorí a	calificaci ón	director
1	Interstell ar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everythin g	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventur e	8.8	C. Nolan

Actores y Actuo_en:

id	nombre	año
1	Leonardo DiCaprio	1974
2	Matthew McConau ghey	1969
3	Daniel Radcliffe	1989
4	Jessica Chastain	1977

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

```
CREATE TABLE Actores(
  id int,
  nombre varchar(30),
  año int
)
```

```
CREATE TABLE
Actuo_en(
  id_actor int,
  id_pelicula int
)
```

Crear tablas con llaves

```
CREATE TABLE Peliculas(
  id int PRIMARY KEY,
  nombre varchar(30),
  año int,
  categoria varchar(30),
  calificación float,
  director varchar(30)
)
```

```
CREATE TABLE Actores(
  id int PRIMARY KEY,
  nombre varchar(30),
  año int
)
```

```
CREATE TABLE Actuo_en(
  id_actor int,
  id_pelicula int,
  PRIMARY KEY (id_pelicula, id_actor)
)
```

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id	nombre	edad	id_actor	id_pelicula
1	Leonardo DiCaprio	41	1	2
2	Matthew McConaughey	46	2	1
3	Daniel Radcliffe	27	4	1
4	Jessica Chastain	39	3	3
			1	5

Valores default

Sintaxis general:

```
CREATE TABLE <Nombre> ( ... <atr> tipo DEFAULT <valor> ... )
```

Ejemplo:

```
CREATE TABLE Peliculas(  
  id int PRIMARY KEY,  
  nombre varchar(30),  
  año int,  
  categoria varchar(30) DEFAULT 'Acción',  
  calificacion float DEFAULT 0,  
  director varchar(30)  
)
```

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Modificar tablas

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Eliminar tabla:

```
DROP TABLE Peliculas
```

Eliminar atributo:

```
ALTER TABLE Peliculas DROP COLUMN director
```

Agregar atributo:

```
ALTER TABLE Peliculas ADD COLUMN productor varchar(30)
```

Insertar datos

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Sintaxis general:

```
INSERT INTO R(at_1, ..., at_n) VALUES (v_1, ..., v_n)
```

Ejemplo:

```
INSERT INTO Peliculas(id, nombre, año, categoria, calificacion, director)  
VALUES (321351, 'V for Vendetta', 2005, 'Action', 8.2, 'James McTeigue')
```

Ejemplo abreviado (asume orden de creación):

```
INSERT INTO Peliculas VALUES (321351, 'V for Vendetta', 2005, 'Action', 8.2, 'James  
McTeigue')
```

Inserciones con llaves foráneas

¿Qué pasa en este caso?

```
CREATE TABLE R(a int, b int, PRIMARY KEY(a));  
CREATE TABLE S(a int, c int, FOREIGN KEY(a)  
REFERENCES R, ...);
```

```
INSERT INTO R VALUES(1, 1);  
INSERT INTO S VALUES(1, 2);
```

Todo bien hasta ahora...

Inserciones con llaves foráneas

¿Qué pasa en este caso?

```
CREATE TABLE R(a int, b int, PRIMARY KEY(a));  
CREATE TABLE S(a int, c int, FOREIGN KEY(a) REFERENCES  
R, ...);
```

```
INSERT INTO R VALUES(1, 1);  
INSERT INTO S VALUES(1, 2);  
INSERT INTO S VALUES(2, 3);
```

ERROR!

La base de datos no permite que se agreguen filas en que la llave foránea no está en la tabla referenciada.

Consulta con SELECT: forma básica

Las consultas en general se ven:

```
SELECT atributos  
FROM relaciones  
WHERE condiciones
```

Para ver todo de una tabla

```
SELECT * FROM Peliculas
```

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Para ver nombre y calificación de todas las películas dirigidas por Nolan:

```
SELECT nombre, calificacion  
FROM Peliculas  
WHERE director = 'C. Nolan'
```

Filtrar con criterios (WHERE)

Para las películas estrenadas desde el 2010:

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

```
SELECT *  
FROM Peliculas  
WHERE año >= 2010
```

WHERE permite =, <>, !=, >, <, <=, >=, AND, OR, NOT, IN, BETWEEN, etc.

IN y BETWEEN

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Películas estrenadas entre 1971 y 1978:

```
SELECT *  
FROM Peliculas  
WHERE año BETWEEN 1971 AND 1978
```

Películas estrenadas en 1971, 1973 y 2001:

```
SELECT *  
FROM Peliculas  
WHERE año IN (1971, 1973, 2001)
```

WHERE permite =, <>, !=, >, <, <=, >=, AND, OR, NOT, IN, BETWEEN, etc.

SELECT en general

La consulta:

```
SELECT a_1, ..., a_n  
FROM T_1, ..., T_m  
WHERE <condicion>
```

Se traduce al álgebra relacional
como:

$$\pi_{a_1, \dots, a_n} (\sigma_{\text{condición}} (T_1 \times \dots \times T_m))$$

- El producto cartesiano combina todas las tablas
- La selección (σ) aplica la condición del WHERE
- La proyección (π) conserva los atributos del SELECT

Actualizar valores (UPDATE)

```
UPDATE Peliculas  
SET calificacion = 0  
WHERE nombre = 'Sharknado 6'
```

Antes

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Sharknado 6	2010	Terror	8.8	A. Ferrante

Después

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Sharknado 6	2010	Terror	0	A. Ferrante

UPDATE: forma general

```
UPDATE R  
SET <Nuevos valores>  
WHERE <Condición sobre R>
```

```
<Nuevos valores> → (atributo1 = nuevoValor1, ..., atributoN =  
                     nuevoValorN)
```

DELETE: forma general

Para borrar filas que cumplan una condición:

```
DELETE FROM R  
WHERE <Condición sobre R>
```

¿Qué pasa si olvidamos el WHERE?

En un UPDATE o DELETE FROM:

```
DELETE FROM Tweets
```

```
UPDATE Users  
SET password = 'contraseña'
```

¿O si se nos pasa un ';' entre medio?

```
UPDATE Users  
SET password = 'contraseña';  
WHERE email = 'some@email.com'
```

```
DELETE FROM Tweets;  
WHERE user_name = 'realDonaldTrump'
```

En ambos casos, ¡se afectan todas las filas de la tabla!

¿Qué pasa si olvidamos el WHERE?

...¿o si se nos pasa un ';' entre medio?

¡Se borran / actualizan todas las tuplas!!

- Siempre verificar la condición WHERE antes de ejecutar
- Considerar el uso de transacciones para poder revertir errores
- Cuidado con separar sentencias con ';' de forma involuntaria

Eliminar con llaves foráneas

Tenemos $S[a] \subseteq R[a]$ (llave foránea)

R	a	b
	1	2
	2	3

S	a	c
	1	3
	2	2

¿Qué ocurre al eliminar (1, 2) en R?

ERROR!

La base de datos no permite eliminar llaves foráneas usadas en otras tablas. Pero hay alternativas.

Eliminar con llaves foráneas

R	a	b
	1	2
	2	3

S	a	c
	1	3
	2	2

¿Qué ocurre al eliminar (1, 2) en R? Tenemos las siguientes opciones:

- No permitir la eliminación
- Propagar la eliminación y también borrar (1,3) de S
- Mantener la tupla en S, pero dejar en la llave foránea el valor NULL

Eliminar con llaves foráneas

Opción 1: no permitir la eliminación (default en SQL)

```
CREATE TABLE R(a int, b int, PRIMARY KEY(a))  
CREATE TABLE S(a int, c int, FOREIGN KEY(a) REFERENCES R, ...)
```

Respuesta: obtenemos error

Eliminar con llaves foráneas

Opción 2: propagar la eliminación (cascada)

```
CREATE TABLE R(a int, b int, PRIMARY KEY(a))  
CREATE TABLE S(a int, c int,  
  FOREIGN KEY(a) REFERENCES R ON DELETE CASCADE, ...)
```

Respuesta: se elimina también (1, 3) en S

Eliminar con llaves foráneas

Opción 3: dejar en NULL

```
CREATE TABLE R(a int, b int, PRIMARY KEY(a))  
CREATE TABLE S(a int, c int,  
  FOREIGN KEY(a) REFERENCES R ON DELETE SET NULL, ...)
```

Respuesta: la tupla (1, 3) en S ahora es (NULL, 3)

Producto cruz

Si pedimos datos de más de una tabla, la base de datos hace un producto cruz y entrega $N \times M$ filas.

```
SELECT * FROM Peliculas, Actuo_en
```

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

Joins

Podemos hacer un JOIN agregando un WHERE. Por ejemplo, para obtener todas las películas junto a los ids de los actores que participaron en ella:

id	nombre	año	categoría	calificación	director	id_actor	id_pelicula
1	Interstellar	2014	SciFi	8.6	C. Nolan	1	2
2	The Revenant	2015	Drama	8.1	A. Iñárritu	2	1
3	Harry Potter	2011	Fantasía	8.1	D. Yates	4	1
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh	3	3
5	El secreto de la selva	2012	Aventura	6.8	C. Nolan	1	5

```
SELECT *  
FROM Peliculas, Actuo_en  
WHERE id = id_pelicula
```

Observación: id es atributo de Peliculas, mientras que id_pelicula es atributo de Actuo_en.

Joins

Desambiguando atributos

id	nombre	año	categoría	calificación	director	id_actor	id_película
1	Interstellar	2014	SciFi	8.6	C. Nolan	1	2
2	The Revenant	2015	Drama	8.1	A. Iñárritu	2	1
3	Harry Potter	2011	Fantasía	8.1	D. Yates	4	1
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh	3	3
5	Inception	2010	Adventure	8.8	C. Nolan	1	5

Entregue todas las películas junto a los id de los actores que participaron en ella:

```
SELECT *  
FROM Peliculas, Actuo_en  
WHERE Peliculas.id = Actuo_en.id_película
```

Sirve cuando tenemos atributos en distintas tablas con el mismo nombre, y agrega claridad a la consulta.

¿Y si queremos los nombres de los actores?

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

id	nombre	edad
1	Leonardo DiCaprio	41
2	Matthew McConaughey	46
3	Daniel Radcliffe	27
4	Jessica Chastain	39

```
SELECT Peliculas.nombre, Actores.nombre
FROM Peliculas, Actuo_en, Actores
WHERE Peliculas.id = Actuo_en.id_pelicula
AND Actores.id = Actuo_en.id_actor
```

Alias

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

id	nombre	edad
1	Leonardo DiCaprio	41
2	Matthew McConaughey	46
3	Daniel Radcliffe	27
4	Jessica Chastain	39

Podemos acortar la consulta anterior:

```
SELECT p.nombre, a.nombre
FROM Peliculas AS p, Actuo_en AS ae, Actores AS a
WHERE p.id = ae.id_pelicula
AND a.id = ae.id_actor
```

Este tipo de alias no es muy recomendable — reduce la legibilidad de la consulta.

Alias en expresiones

Podemos hacer operaciones y nombrar la columna resultante:

```
SELECT (nombre || ' dirigida por ' || director) AS credits, año  
FROM Peliculas
```

credits	año
Interstellar dirigida por C. Nolan	2014

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Ordenando (ORDER BY)

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Entregue el nombre y la calificación de todas las películas (orden ascendente):

```
SELECT nombre, calificacion
FROM Peliculas
ORDER BY nombre, calificacion
```

El i-ésimo atributo del ORDER BY resuelve un empate en el atributo i-1.

```
ORDER BY nombre DESC, calificacion
```

Operadores de conjuntos

EXCEPT

Diferencia del álgebra relacional

UNION

Unión del álgebra relacional

INTERSECT

Intersección del álgebra relacional

UNION ALL

Unión que admite duplicados

UNION

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id	nombre	edad
1	Leonardo DiCaprio	41
2	Matthew McConaughey	46
3	Daniel Radcliffe	27
4	Jessica Chastain	39

Entregue el nombre de todos los actores y directores:

```
SELECT nombre FROM Actores
UNION
SELECT director FROM Peliculas
```

Matching de patrones

con LIKE

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

s LIKE p: string s “es como” p, donde p es un patrón definido mediante:

- % → cualquier secuencia de caracteres
- _ → cualquier carácter (solamente uno)

```
SELECT * FROM Peliculas WHERE nombre LIKE '%Potter%'
```

Eliminando duplicados

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

Entregue todos los nombres distintos de las películas:

```
SELECT DISTINCT nombre FROM Peliculas
```

OJO: DISTINCT es un operador en sí mismo.

Agregación

¿Qué hace esta consulta?

```
SELECT AVG(precio)
FROM Productos
WHERE fabricante = 'Toyota'
```

Entrega el precio promedio de los productos marca Toyota

También podemos usar SUM, MIN, MAX, COUNT, etc.

Agregación: COUNT(*) vs. COUNT(atributo)

¿Qué diferencia estas consultas?

```
SELECT COUNT(*)  
FROM Productos  
WHERE año > 2012
```

```
SELECT COUNT(fabricante)  
FROM Productos  
WHERE año > 2012
```

- **COUNT(*)** sí cuenta los nulos
- En ambas se cuentan los duplicados

Ejemplo: COUNT

Compra(producto, fecha, precio, cantidad)

producto	fecha	precio	cantidad
tomates	07/02	100	6
tomates	06/07	150	
zapallo	08/02	800	1
zapallo	09/07	1000	
zapallo	01/01	600	1

```
SELECT COUNT(*)  
FROM Compra
```

5

```
SELECT COUNT(cantidad)  
FROM Compra
```

3

Ejemplo: DISTINCT + COUNT

Compra(producto, fecha, precio, cantidad)

producto	fecha	precio	cantidad
tomates	07/02	100	6
tomates	06/07	150	
zapallo	08/02	800	1
zapallo	09/07	1000	
zapallo	01/01	600	1

```
SELECT DISTINCT COUNT(cantidad)  
FROM Compra
```

3

```
SELECT COUNT(DISTINCT cantidad)  
FROM Compra
```

2

GROUP BY

```
SELECT fabricante, COUNT(fabricante)
      FROM Productos
     WHERE año > 2012
    GROUP BY fabricante
```

Esta consulta:

- Computa los resultados según el FROM y WHERE
- Agrupa los resultados según los atributos del GROUP BY
- Para cada grupo se aplica independientemente la agregación

Ejemplo: GROUP BY

Compra(producto, fecha, precio, cantidad)

```
SELECT producto, SUM(precio*cantidad) AS ventaTotal  
FROM Compra WHERE fecha > '10/01' GROUP BY producto
```

1) FROM + WHERE

producto	fecha	precio	cantidad
tomates	07/02	100	6
tomates	06/07	150	4
zapallo	08/02	800	1
zapallo	09/07	1000	2
zapallo	04/04	600	3

2) Agrupar por producto según Group By

producto	fecha	precio	cantidad
tomates	07/02	100	6
	06/07	150	4
zapallo	08/02	800	1
	09/07	1000	2

3) Agregar por grupo y proyectar

producto	ventaTotal
tomates	1200
zapallo	2800

HAVING

Misma consulta, pero sólo queremos los productos que se vendieron más de 100 veces:

```
SELECT producto, SUM(precio*cantidad) AS ventaTotal  
FROM Compra  
WHERE fecha > '10/01'  
GROUP BY producto  
HAVING SUM(cantidad) > 100
```

¿Por qué usamos HAVING y no lo incluimos en el WHERE?

Consultas con agregación

```
SELECT <S>  
FROM R1, ..., Rn  
WHERE <Condición 1>  
GROUP BY a1, ..., ak  
HAVING <Condición 2>
```

- S puede contener atributos de a1, ..., ak y/o agregados, pero ningún otro atributo
- Condición 1 usa atributos de R1, ..., Rn
- Condición 2 es una condición de agregación sobre los atributos de R1, ..., Rn

Evaluación de la consulta

```
SELECT <S>  
FROM R1, ..., Rn  
WHERE <Condición 1>  
GROUP BY a1, ..., ak  
HAVING <Condición 2>
```

- Se computa el FROM - WHERE de $R_1, \dots, R_n$
- Se agrupa la tabla por los atributos $a_1, \dots, a_k$
- Se computan los agregados de Condición 2 y se mantienen los grupos que la satisfacen
- Se computan los agregados de S y se entrega el resultado

Consultas anidadas

- Como ya vimos con las operaciones de conjuntos, una consulta puede estar constituida por operaciones entre consultas
- Pero esa no es la única forma: SQL nos ofrece mucho más

Consultas anidadas

Bandas(nombre, vocalista, ...)

Estudiantes_UC(nombre, ...)

Toco_en(nombre_banda, nombre_festival)

Obtengamos todas las bandas cuyo vocalista sea estudiante UC y que hayan tocado en Lollapalooza:

```
SELECT Bandas.nombre
FROM Bandas, Estudiantes_UC
WHERE Bandas.vocalista = Estudiantes_UC.nombre
AND Bandas.nombre IN (
    SELECT Toco_en.nombre_banda
    FROM Toco_en
    WHERE Toco_en.nombre_festival = 'Lollapalooza'
)
```

Consultas anidadas

Si la sub-consulta retorna un escalar, podemos usar los operadores condicionales típicos. Si no, agregamos un operador adicional:

S IN R

S > ALL R

S > ANY R

EXISTS R

Consultas anidadas: ALL, ANY

Cervezas(nombre, precio, ...)

Cervezas más baratas que la Austral Calafate:

```
SELECT Cervezas.nombre
FROM Cervezas
WHERE Cervezas.precio < ALL (
    SELECT Cervezas2.precio
    FROM Cervezas AS Cervezas2
    WHERE Cervezas2.nombre = 'Austral Calafate'
)
```

Consultas anidadas: ALL, ANY

Cervezas que no son la más cara:

```
SELECT Cervezas.nombre  
FROM Cervezas  
WHERE Cervezas.precio < ANY (  
    SELECT Cervezas2.precio  
    FROM Cervezas AS Cervezas2  
)
```

Consultas anidadas

¿Podemos expresar estas consultas con **SELECT - FROM - WHERE**?

Hint: Las consultas SFW son monótonas¹. Una consulta con ALL no es monótona. Una consulta con ANY sí lo es.

¹Una consulta es "monótona" si agregar más datos nunca reduce el conjunto de resultados que produce.

Sub-consultas relacionadas

Nombres de películas que se repiten en años diferentes:

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

```
SELECT p.nombre
FROM Peliculas AS p
WHERE p.año <> ANY (
    SELECT año FROM Peliculas
    WHERE nombre = p.nombre
)
```

La sub-consulta depende de la externa (correlacionada).

Consultas anidadas como joins

El nombre de cada actor junto con el total de películas en las que ha actuado:

id	nombre	edad
1	Leonardo DiCaprio	41
2	Matthew McConaughey	46
3	Daniel Radcliffe	27
4	Jessica Chastain	39

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

```
SELECT Actores.nombre, agg.cuenta
FROM Actores, (
    SELECT id_actor AS id, COUNT(*) AS cuenta
    FROM Actuo_en
    GROUP BY id_actor
) AS agg
WHERE Actores.id = agg.id
```

Consultas anidadas como joins

El nombre de cada actor junto con el año de su primera película:

```
SELECT Actores.nombre, agg.año
FROM Actores, (
    SELECT Actuo_en.id_actor AS id, MIN(Peliculas.año) AS año
    FROM Actuo_en, Peliculas
    WHERE Actuo_en.id_pelicula = Peliculas.id
    GROUP BY id_actor
) AS agg
WHERE Actores.id = agg.id
```

¿Qué pasa si queremos, además, el nombre de la película?

Información incompleta

- En una base de datos real, muy seguido no tendremos los datos para llenar todas las columnas al agregar una fila
- También puede ser que, por la lógica del problema, un campo vacío tenga una semántica relevante para la aplicación
- **Con SQL podemos modelar la falta de información mediante nulos (NULL)**
- Los nulos generan comportamientos extraños que conviene tener en cuenta — los discutiremos en esta clase

Información incompleta

Películas

nombre	director	actor
Django sin cadenas	Tarantino	Di Caprio
Django sin cadenas	Tarantino	Waltz
NULL	Tarantino	Thurman
NULL	Tarantino	NULL
El Hobbit	Jackson	McKellen
Señor de los Anillos	NULL	McKellen

¿Qué significan los nulos en este caso?

Significado de los nulos

En el caso anterior puede significar que no se dispone de la información, ¡pero existe!

En general, los nulos pueden significar:

- El valor existe, pero no tenemos la información
- El valor no existe (si nombre = NULL, la información no existe)
- Ni siquiera sabemos si el valor existe o no

Consultando con nulos

Sea la relación $R(a, b)$, las consultas:

```
SELECT * FROM R
```

```
SELECT * FROM R WHERE R.b = 3 OR R.b <> 3
```

¿Son lo mismo?

Si $R.b$ es nulo, ni $R.b = 3$ ni $R.b <> 3$ evalúan a verdadero.

Consultando con nulos

La consulta `SELECT * FROM R` equivale a la unión de:

```
SELECT * FROM R WHERE R.b = 3  
SELECT * FROM R WHERE R.b <> 3  
SELECT * FROM R WHERE R.b IS NULL
```

IS NULL → para ver si un elemento es nulo

IS NOT NULL → para ver si un elemento no es nulo

Operaciones con nulos

Si algún argumento de una operación aritmética es nulo, el resultado es nulo.

R	A
	1
	NULL

S	B
	2
	3

SELECT R.A + S.B FROM R, S retorna:

R.A + S.B
3
4
NULL

Operaciones con nulos

R	A
	1
	NULL

S	B
	2
	3

Tenemos que $R.A = S.B$ vale:

FALSE cuando $R.A = 1$ y $S.B = 2$

UNKNOWN cuando $R.A = \text{NULL}$ y $S.B = 2$

Lógica de tres valores

SQL usa lógica de tres valores: TRUE, FALSE y UNKNOWN.

x	NOT x
true	false
false	true
unknown	unknown

Ejemplo: NOT IN

R	A
	1
	2

S	A
	1
	2
	3
	4

```
SELECT S.A FROM S  
WHERE S.A NOT IN (SELECT R.A FROM R)
```

Resultado: {3, 4}

Ejemplo: NOT IN con NULL

R	A
	1
	2
	NULL

S	A
	1
	2
	3
	4

```
SELECT S.A FROM S  
WHERE S.A NOT IN (SELECT R.A FROM R)
```

Resultado: tabla vacía

Lógica de tres valores

El resultado puede ser contraintuitivo, pero es correcto. Veamos la evaluación de:

```
3 NOT IN (SELECT R.A FROM R)
```

- $3 = 1 \rightarrow \text{FALSE}$, $3 = 2 \rightarrow \text{FALSE}$, $3 = \text{NULL} \rightarrow \text{UNKNOWN}$
- $3 \text{ NOT IN } R$ se evalúa como $\text{NOT}(\text{FALSE OR FALSE OR UNKNOWN})$
- $= \text{NOT}(\text{UNKNOWN}) = \text{UNKNOWN}$

Lógica de tres valores

- Además, 1 NOT IN (SELECT R.A FROM R) es FALSE (lo mismo para 2)
- Para el valor 4 aplicamos el razonamiento anterior

Resultado: tabla vacía

Agregación con nulos

R	A
	1
	NULL

```
SELECT COUNT(*) FROM R      vale  2
SELECT COUNT(R.A) FROM R    vale  1
```

Agregación con nulos

```
SELECT SUM(R.A) FROM R  retorna 1  si  R.A = {1, NULL}
```

Para funciones de agregación:

- Se ignoran todos los nulos
- Se computa el valor de la agregación
- **La única excepción es COUNT(*)**

Impedir nulos

Al crear una tabla o agregar una columna, podemos incluir una restricción para que no permita NULLs:

```
CREATE TABLE <nombre> (  
    <atributo> <tipo> NOT NULL,  
    ...  
)
```

Inner Joins

- Usualmente hacemos joins especificando en el FROM las tablas y en el WHERE las condiciones

```
SELECT *  
FROM Peliculas, Actuo_en  
WHERE id = id_pelicula
```

**SQL tiene una sintaxis mucho más clara para expresar un
join normal:
INNER JOIN (alias JOIN)**

Inner Joins

Estas 3 consultas son equivalentes

```
SELECT * FROM Peliculas, Actuo_en  
WHERE id = id_pelicula
```

```
SELECT * FROM Peliculas JOIN Actuo_en  
ON id = id_pelicula
```

```
SELECT * FROM Peliculas INNER JOIN Actuo_en  
ON id = id_pelicula
```

id	nombre	año	categoría	calificación	director
1	Interstellar	2014	SciFi	8.6	C. Nolan
2	The Revenant	2015	Drama	8.1	A. Iñárritu
3	Harry Potter	2011	Fantasía	8.1	D. Yates
4	The Theory of Everything	2014	Biografía	7.7	J. Marsh
5	Inception	2010	Adventure	8.8	C. Nolan

id_actor	id_pelicula
1	2
2	1
4	1
3	3
1	5

Outer Joins

Consideremos estas tablas. Escribamos una consulta que liste los ingresos totales de cada estudio:

Estudios

nombre	titulo
Warner	Argo
Warner	El Origen
MGM	El Hobbit

Peliculas

titulo	ingreso
Argo	136
El Origen	292
El Artista	44

Outer Joins

```
SELECT Estudio.nombre, SUM(Pelicula.ingreso)
FROM Estudio JOIN Pelicula
ON Estudio.titulo = Pelicula.titulo
GROUP BY Estudio.nombre
```

Resultado: (Warner, 428)

¿Cuál es el problema de esta consulta?

El estudio MGM, cuyas películas no tenemos, va a desaparecer por no tener contraparte en el JOIN.

Outer Joins

Lo solucionamos con un LEFT OUTER JOIN

Mantiene las tuplas sin pareja de la primera tabla:

```
SELECT Estudio.nombre, SUM(Pelicula.ingreso)
FROM Estudio LEFT OUTER JOIN Pelicula
ON Estudio.titulo = Pelicula.titulo
GROUP BY Estudio.nombre
```

Resultado: (Warner, 428), (MGM, NULL)

Left Outer Join

```
SELECT *  
FROM Estudio LEFT OUTER JOIN Pelicula  
ON Estudio.titulo = Pelicula.titulo
```

nombre	titulo	titulo	ingreso
Warner	Argo	Argo	136
Warner	El Origen	El Origen	292
MGM	El Hobbit	El Hobbit	NULL

Outer Joins

R LEFT OUTER JOIN S

Mantenemos las tuplas de R que no tienen correspondencia

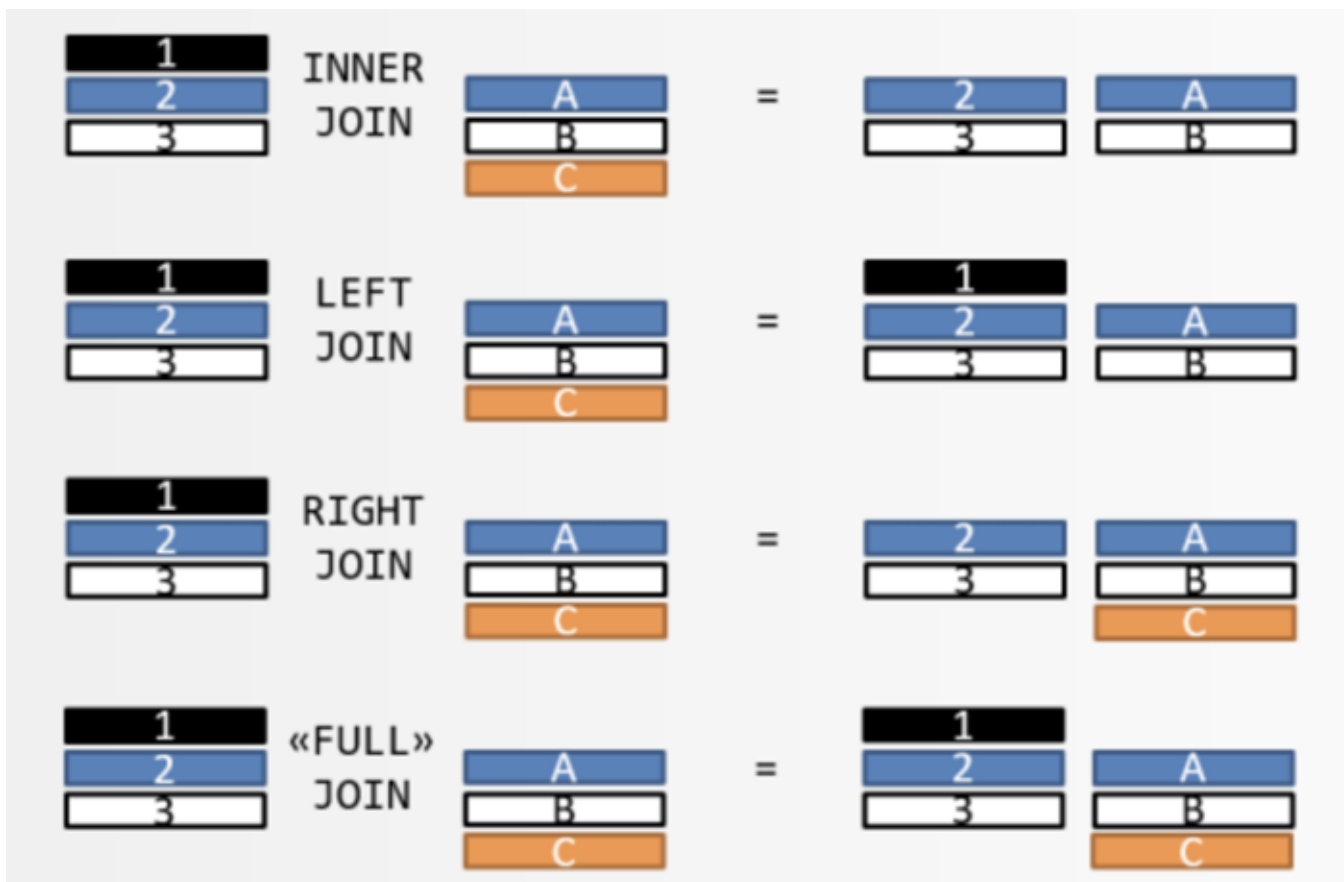
R RIGHT OUTER JOIN S

Mantenemos las tuplas de S que no tienen correspondencia

R FULL OUTER JOIN S

Mantenemos las tuplas de R y S que no tienen correspondencia

Resumen visual de Joins



Redundancia en SQL

Recordemos esta consulta

Bandas(nombre, vocalista, ...)

Estudiantes_UC(nombre, ...)

Toco_en(nombre_banda, nombre_festival)

```
SELECT Bandas.nombre
FROM Bandas, Estudiantes_UC
WHERE Bandas.vocalista = Estudiantes_UC.nombre
AND Bandas.nombre IN (
    SELECT Toco_en.nombre_banda
    FROM Toco_en
    WHERE Toco_en.nombre_festival = 'Lollapalooza'
)
```

Redundancia en SQL

¿Cómo saber cuál usar?

- Dada la naturaleza declarativa de SQL, es muy difícil predecir cómo distintas formas de escribir una consulta afectan el rendimiento al ejecutarse en el RDBMS
- Una aplicación real necesita hacer muchas consultas junto con código procedural; cada consulta implica conectarse a la base de datos, y las conexiones tienen un overhead de tiempo adicional
- **En la práctica, generalmente optimizamos el número de consultas a hacer, más que cómo están escritas esas consultas**

Tres formas equivalentes

```
SELECT Bandas.nombre
FROM Bandas, Estudiantes_UC
WHERE Bandas.vocalista = Estudiantes_UC.nombre
AND Bandas.nombre IN (
    SELECT Toco_en.nombre_banda FROM Toco_en
    WHERE Toco_en.nombre_festival = 'Lollapalooza')
```

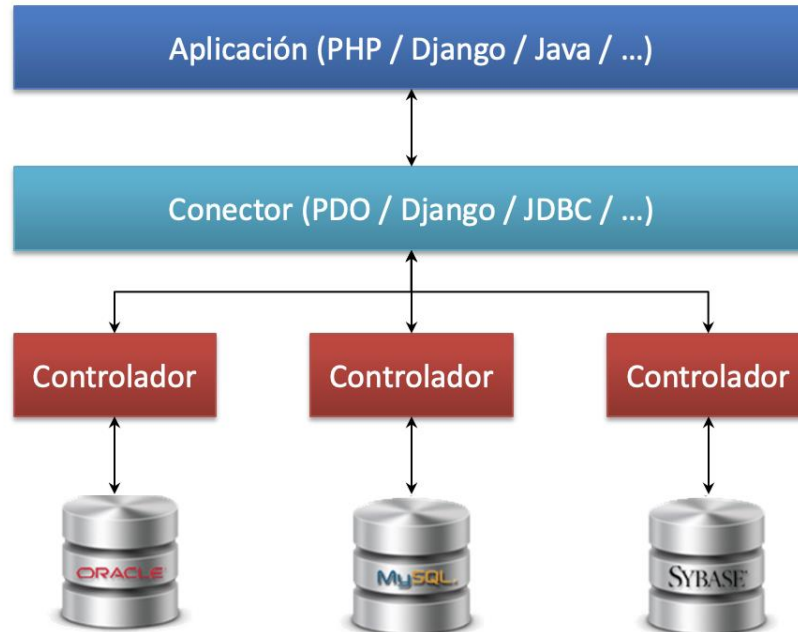
```
SELECT DISTINCT Bandas.nombre
FROM Bandas, Estudiantes_UC, Toco_en
WHERE Bandas.vocalista = Estudiantes_UC.nombre
AND Bandas.nombre = Toco_en.nombre_banda
AND Toco_en.nombre_festival = 'Lollapalooza'
```

```
SELECT Bandas.nombre
FROM Bandas, Estudiantes_UC
WHERE Bandas.vocalista = Estudiantes_UC.nombre
INTERSECT
SELECT Toco_en.nombre_banda FROM Toco_en
WHERE Toco_en.nombre_festival = 'Lollapalooza'
```

¡Son todas equivalentes!

Programación y SQL

Acceso programático



Una aplicación no habla SQL directamente con RDBMS: pasa siempre por un conector/driver que traduce las llamadas del lenguaje al protocolo del RDBMS.

Definir conector y consultar

Conectores desde PHP

PHP / PostgreSQL

```
$conn = pg_connect("host = ... port = ... dbname = ... user = ... password = ...");
$result = pg_query($conn, "SELECT atr FROM tabla WHERE ...");
if($result){
    while ($row = pg_fetch_assoc($result)){
        echo "Result is ". $row["atr"];
    }
}
```

PHP / PHP Data Objects (PDO)

```
$conn = new PDO("pgsql:host=...;port=...;dbname=...;user=...;password=...");
$stmt = $conn->query("SELECT atr FROM tabla WHERE ...");
while ($row = $stmt->fetch()){
    echo "Result is ". $row["atr"];
}
```

PDO ofrece más flexibilidad para cambiar el sistema de bases de datos.

Cursores

Para ejecutar comandos SQL desde PHP, debemos usar cursores.

```
$stmt = $pdo->query('SELECT * FROM Peliculas');  
while ($fila = $stmt->fetch()) {  
    echo $fila['nombre'];  
}
```

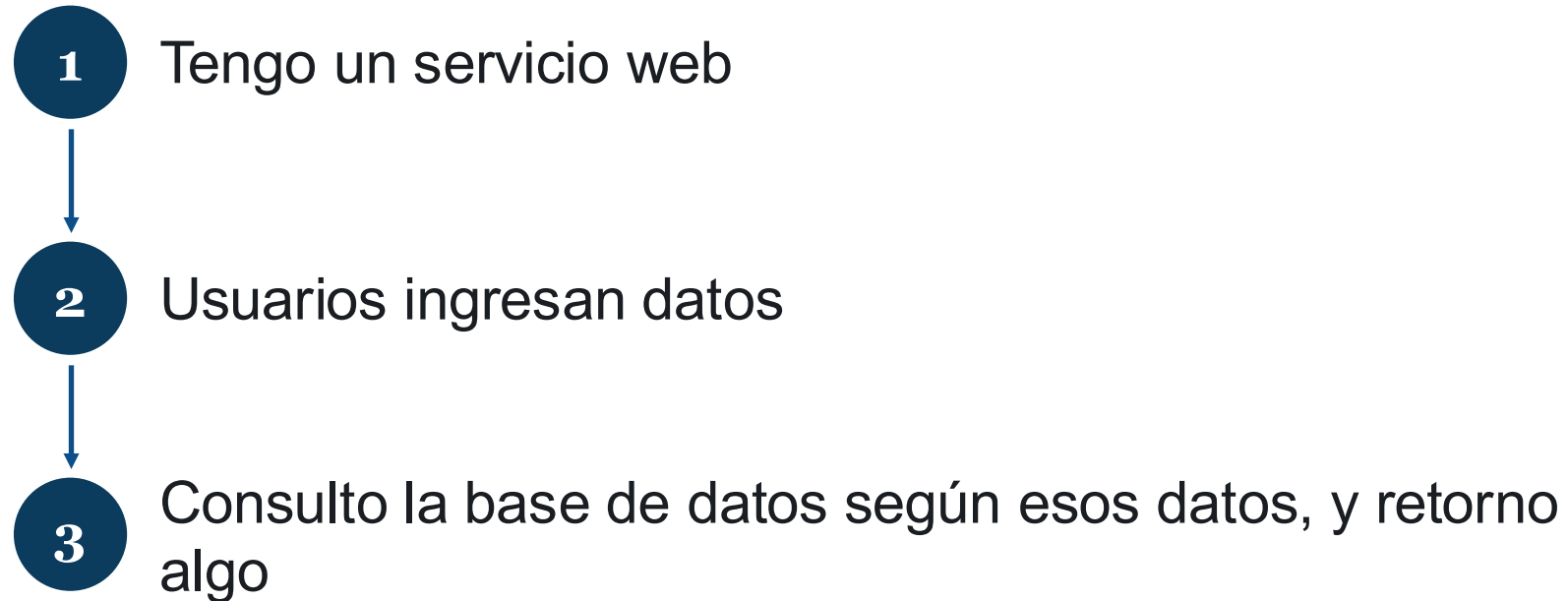
Cursores

Sabemos hacer consultas, ¿pero cómo le entregamos los resultados al lenguaje de programación?

Los cursores nos entregan los resultados de una fila a la vez.

¿Por qué no conviene entregar todos los resultados de una sola vez?

Casos de uso más relevantes



SQL parametrizado

- Se trata de preparar una consulta con variables
- Cuando conozco el valor de las variables, se instancia la nueva consulta concreta
- A su debido tiempo, la ejecuto

```
$stmt = $pdo->prepare('SELECT * FROM Peliculas WHERE director = :dir');  
$stmt->execute(['dir' => $nombreDirector]);
```

¿Por qué no concatenar?

- Concatenar directamente el input del usuario dentro de una consulta SQL permite inyección de código (SQL injection)
- Si el usuario ingresa código SQL malicioso como parte de un dato (p. ej. un nombre), este puede ejecutarse sin querer
- *Ejemplo clásico: xkcd.com/327 — un colegio con un alumno llamado "Robert"); DROP TABLE Students;--"*

El SQL parametrizado evita este problema: el driver trata cada valor como dato, nunca como código ejecutable.

¡Gracias!

Bases de Datos · Clase 6 y 7: SQL